Takeaway Order Dispatch System

Note

- This is an individual assignment. You are not allowed to copy other students' solutions or share your own solution with other students. Cheating and plagiarism will be caught and punished HEAVILY! https://registry.hkust.edu.hk/resource-library/what-happens-if-you-are-caught-cheating
- Use of generative AI tools (such as ChatGPT and Copilot) is allowed, but its use must be properly acknowledged, and the **prompts must be clearly documented as inline comments**.
- Download the project skeleton from Canvas and complete the tasks based on it. The main method of `DispatchSystem.java` in the skeleton is the entrance for running the assignment.
- The due time is 11:59 pm on **October 12**. Compress only the `src` folder in the project directory as a zip file and **submit it on Canvas**. Write the GitHub repository link as a comment for the Canvas submission.
- Your GitHub repository must be **private** and have the TA account (https://GitHub.com/COMP3021-2024Fall) invited as a collaborator. The GitHub repository must **have at least three commits separated in two days**, to prevent obvious plagiarism. The final commit on the repository should contain at least the same files as the submitted files. Failing this will result in 20% mark penalty on top of other penalties.
- Late submissions (regardless of technical issues such as Canvas or GitHub downtime) will be penalized by 20% of the total attainable marks in this assignment per day. This assignment accounts for **16% of the total course grade**.
- For general questions about the assignment, please ask in the "student-discussion" channel on Discord or email all TAs (the TA emails are listed on Canvas), instead of sending private messages to each TA individually. If your question includes code snippets in your solution, please email the TAs rather than posting it on Discord. Last minute questions on the day of the assignment/lab deadline will be ignored, unless it is part of an ongoing conversation. In general, you should ask questions regarding labs and assignments **at least 3 days before the deadline**.

1 Prerequisites, Goals, and Outcomes

1.1 Prerequisites

For this programming assignment, you should have learned:

- Knowledge of class design: class attributes, constructors, accessor methods, mutator methods, abstract classes, and interface.
- Knowledge of inheritance and polymorphism: specialization, generalization, dynamic binding, and generic programming.
- Use of Java collections: Lists, Maps, and iterators.

1.2 Goals

1. Practise Java OOP principles, especially inheritance and interfaces.
2. Understand the importance of data management and operations using collections.

1.3 Outcomes

Upon completion, you should be able to:

1. Design and implement Java classes and their constructors, accessors, and mutators.
2. Use inheritance and interfaces effectively to achieve polymorphism.
3. Develop classes that harness the power of Java collections for complex data operations.

2 Task Description

In this assignment, you will design and implement a takeaway order dispatch system.

2.1 Part 0: Takeaway Order Dispatch system structure (10 marks)

Table 1, Table 2, and Table 3 contain the description of the important entities in this system. Some of them are already (partially) provided in the skeleton, and others are not provided. You are required to declare and/or implement all of the attributes and methods as listed in these three tables. The names and types of these attributes and methods must be exactly the same as in the tables. This is important because the automated grading tool will be accessing some of these methods.

You are allowed to define additional attributes and methods that are not listed in the tables to support your own implementation.

To get full 10 marks for Part 0, your system should: (1) define the classes listed in Table 1, Table 2, and Table 3 with the same names for public classes, methods, and attributes, (2) Define and implement the *DispatchSystem* class using the Singleton design pattern (see below) and (3) compile successfully.

With the Singleton design pattern, the constructor for the *DispatchSystem* class should have the following behavior:

- If no instances of this class have been created before, then the constructor should initialize all of the member fields for later use.

- If any instance of this class has already been created before, then the constructor should throw an exception.

The `getInstance()` method for the *DispatchSystem* class should have the following behavior:

- If no instances of this class have been created before, then the `getInstance()` method should create a new instance and store it properly for later use.
- If any instance of this class has already been created before, then the `getInstance()` method should return the created instance.

Task 1: Implement the constructor and the `getInstance()` method for the *DispatchSystem* class. This class can only be used as singleton in this project as described above.

2.2 Part 1: Data Modeling and Preparation (30 marks)

In this section, you should process the input files and convert them into data structures in your program. These data structures will be used later for dispatching orders and for data analytics.

Task 2: The *Account* class has a protected inner class *AccountManager* and a static instance of *AccountManager* class. You should implement the abstract method `register()` for all child classes of the *Account* class (Customer, Restaurant, and Rider) so that the `register()` method inserts each account instance to the corresponding container fields inside the *AccountManager* class. Note that, because the fields are private, to read and write these private fields, you need to first implement several public methods that enable external accesses. Next, implement the `parseAccounts()` method to parse the accounts from the input file "SampleIntputAccount.txt" and convert it into a data structure in the program. See Section 3 for the meanings of the input format. Then, store each created account into the static *AccountManager* instance.

Task 3: Implement the `parseDishes()` method to parse the dishes from the input file "SampleIntputDishes.txt". You will need to use the RestaurantId field (see Section 3) to identify the corresponding restaurant. Do not forget to add the created dishes into the corresponding restaurant and the *availableDishes* field.

Task 4: Implement the `parseOrders()` method to parse the orders from the file "SampleIntputOrders.txt". Do not forget to add the order to the *availableOrders* field. You should check if all the dishes listed in the same order (see the OrderedDishes field in Section 3) are offered by the same restaurant. If not, then treat this order as illegal and skip this line of record without adding it to the *availableOrders* field. You may need to implement and use some auxiliary methods here, e.g. `getDishById()`, `getCustomerById()`, and `checkDishesInRestaurant()`.

Sample input and output for Part 1: When your program processes the input files SampleIntputAccounts.txt, SampleIntputDishes.txt, and SampleIntputOrders.txt (a sample of these files are provided in the skeleton), your `availableOrders` field should contain all the legal orders. Your program should then output the `availableOrders` list to a availableOrders.txt file using the provided `writeOrders()` method.

For the provided sample input files, the correct output for Part 1 should be identical to the SampleOutputAvailableOrders.txt file provided in the skeleton.

To get the full $10 + 30 = 40$ marks for both Part 0 and Part 1, your program should: (1) successfully parse the provided sample input files, (2) produce the correct output file from the `writeOrders()` method, and (3) pass a set of hidden tests for Part 1.

2.3 Part 2: Dispatch orders to riders (50 marks)

In this part, you should use the following strategy to dispatch the currently available orders to the most appropriate rider. The text below mentions some capitalized constants, whose values are defined in the `Constants.java` file in the skeleton.

Detailed rules of the dispatch strategy:

1. Each rider can only deliver one takeaway at a time and each order can only be dispatched to one rider one time.

2. In the order dispatch process, your program should keep dispatching orders to riders following a specific ranking criterion (see 5 and 6 for detail) until there are no more orders or riders.

3. Each rider may have one of three statuses at a time: *RIDER_ONLINE_ORDER* (indicates the rider is available and ready to receive a new order), *RIDER_DELIVERING* (indicates the rider has already received an order and is currently delivering, so they are not ready to receive other orders), and *RIDER_OFFLINE* (indicates the rider is offline and not ready to receive any orders).

4. Each order may have one of three statuses at a time: *PENDING_ORDER* (indicates the order is waiting to be dispatched to a rider), *DISPATCHED_ORDER* (indicates the order has already been dispatched to a rider and is being delivered), and *COMPLETED_ORDER* (indicates the order has been delivered and archived).

5. The orders should be dispatched following a specific ranking:

   - Top priority: Customer type. VIP's orders should be dispatched earlier than non-VIP's orders.
   - Second priority: Order creation time. For customers of the same type, the earlier an order is created, the sooner the order is dispatched.
   - Third priority: Distance. For customers of the same type and the same creation time, the farther the distance is between the restaurant and the customer (see Part 3 for how to calculate distances), the sooner the order should be dispatched.

6. In the process of matching the best rider for a specific order, you should first construct a list of tasks (see the *Task* class in Table 2) where each task consists of the specific order and a potential rider. This list should contain one task for each currently available rider. Then, you should rank this list of tasks following a specific ranking:

   - Top priority: Distance. The closer the distance (see Part 3) between the rider and the restaurant is, the higher the task is ranked.
   - Second priority: Rider's rating. The higher the rating is, the higher the task is ranked.
   - Third priority: Rider's monthly task count. The fewer the rider's monthly task count is, the higher the task is ranked.

   After the list of tasks is correctly ranked, you should select the highest-ranked task as the result of dispatching the order.

Task 5: Implement the `getAvailablePendingOrders()` method to get the available pending orders that can be dispatched this time, referring to the description in 4. And the order should be paid and has no rider dispatched.

Task 6: Implement the `getRankedPendingOrders()` method to rank the pending orders by the strategy described in 5. You should implement the Comparators described in Table 3. In particular, the skeleton has already provided the *PendingOrderRank* and *TaskRank* interfaces. You should define and implement the other classes listed in the table. Each such class should implement one of the two interfaces.

Task 7: Implement the `getAvailableRiders()` method to get the available riders that can be dispatched for the first round, referring to 3.

Task 8: Implement the `matchTheBestTask()` method to choose the best rider for the order. The `matchTheBestTask()` method takes an order and a list of **currently available** riders as input. This method should use the list of riders, along with the order, to construct tasks (see Table 2) and then rank the tasks. The highest-ranked task means the best match of the order and the rider, referring to 6 for details. After selecting the top-match rider, Task 9 will ask you to **remove** this top-match rider from the currently available riders for later use.

Task 9: Implement the `dispatchFirstRound()` method to dispatch the currently pending orders, referring to 1 and 2. Do not forget to update the status, rider, estimated time of the order and the status of the rider. And remove the top-match rider described in Task 8, such that the top-match rider will no longer appear in future invocations to the `matchTheBestTask()` method.

You should complete the tasks above so that invoking the `dispatchFirstRound()` method of the *DispatchSystem* class completes the dispatch process.

Sample output for Part 2: This part takes as input the data structures parsed in Part 1. When your program finishes the dispatch process, you should get a list of dispatched orders stored in `dispatchedOrders` field. You should output this field to a firstRoundDispatchedOrders.txt file using the provided `writeOrders()` method. Refer to section 3 for details of the input and output formats.

For the provided sample input files (see Part 1), the correct output for Part 2 should be identical to the SampleOutputFirstRoundDispatchedOrders.txt file provided in the skeleton.

To get full 50 marks for Part 2, your program should: (1) produce the correct list of orders from the `dispatchFirstRound()` method and successfully dump it to a file using `writeOrders()` method, (2) pass a set of hidden tests for Part 2.

2.4 Part 3: Data analytics (10 marks)

In this section, you should implement the following methods in the *DispatchSystem* to analyze which **dispatched** orders will be "timeout".

In our takeaway order dispatch system, we use the following criteria to calculate the estimated delivery time for each order:

1. We assume that the current time stamp (see `currentTimestamp` in the skeleton code) is 3600 (unit: seconds). You may assume that the current orders will always have created time stamps less than 3600.

2. The rider's delivery speed (see `Constant.java` file in the skeleton code) is 4 m/s.

3. The locations of riders, restaurants, and customers are represented by a tuple (latitude, altitude), e.g. [100, 200] (unit: meters). And thus the distance between two locations, $d([x_1, y_1], [x_2, y_2])$ is calculated by the formula: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

4. The estimated delivery time is calculated by adding the estimated duration on the road (see 2 and 3) to the current timestamp (see 1).

Task 10: Implement the `getTimeoutDispatchedOrders()` method to get the "timeout" dispatched orders, which is judged by checking if the time elapsed from the order's creation time to its estimated delivery time exceeds the time limit of 1800 seconds, which is provided in the Constants.java file.

Sample input and output for Part 3: This part takes as input the data structures parsed in Part 1. Your `getTimeoutDispatchedOrders()` method should generate a list of orders. You should output this list to a timeoutDispatchedOrders.txt file using the provided `writeOrders()` method. Refer to section 3 for details of the input and output formats.

For the provided sample input files (see Part 1), the correct output for Part 3 should be identical to the SampleOutputTimeoutDispatchedOrders.txt file provided in the skeleton.

To get full 10 marks for Part 3, your program should: (1) produce the correct list of orders from the `getTimeoutDispatchedOrders()` method and successfully dump it to a file using `writeOrders()` method, and (3) pass a set of hidden tests for Part 3.

3 Input and Output

Your program takes three `txt` files as input.

3.1 Input

Each line in Accounts.txt represents an account and contains three types:

Customer

- Name: Id, AccountType, Name, ContactNumber, Location, CustomerType, Gender, Email.
- Type: Long, String, String, String, List<Double>, Integer, String, String.
- Example: 1, CUSTOMER, Alice Johnson, 23457890, [35 45], 2, Female, alice.johnson@example.com

Restaurant

- Name: Id, AccountType, Name, ContactNumber, Location, District, Street
- Type: Long, String, String, String, List<Double>, Integer, String, String
- Example: 22, RESTAURANT, Taco Stop, 89876234, [1005 295], SaiKung District, Street B

Rider

- Name: Id, AccountType, Name, ContactNumber, Location, Gender, Status, UserRating, MonthTaskCount
- Type: Long, String, String, String, List<Double>, String, Integer, Double, Integer
- Example: 43, RIDER, Emily C, 89891125, [1145 435], Female, 4, 7, 100

Each line in Dishes.txt represents a course and contains:

- Name: Id, Name, Desc (The description contains no commas), Price, RestaurantId
- Type: Long, String, String, BigDecimal, Long
- Example: 1, Taco, "A traditional Mexican dish", 30.5, 22

Each line in Orders.txt represents a course and contains:

- Name: Id, Status, RestaurantId, CustomerId, CreateTime, IsPayed, OrderedDishes, RiderId (NA for not applicable, assign null to rider field in this case)
- Type: Long, Integer, Long, Long, Long, Integer (0 or 1), List<Long>, Long or NA
- Example: 1, 6, 32, 10, 2700, 1, [12 54 33], NA

3.2 Output

After the enrollment process, you should provide:

- availableOrders.txt: Just use the `writeOrders()` method.
- firstRoundDispatchedOrders.txt: Just use the `writeOrders()` method.
- imeoutDispatchedOrders.txt: Just use the `writeOrders()` method.
  - Name: Id, Status, Restaurant, Customer, CreateTime, IsPayed, OrderedDishes, Rider, EstimatedTime
  - Type: Long, Integer, Restaurant, Customer, Long, Boolean, List<Dish>, Rider, Double
  - Example: 1, 6, Restaurantid=32, accountType='RESTAURANT', name='French Cafe', contactNumber='89875789', location=Location(latitude=1100.0, altitude=390.0), district='SaiKung District', street='Street L', dishIds='[12, 33, 54, 75, 96]', Customerid=10, accountType='CUSTOMER', name='Julia Taylor', contactNumber='23457811', location=Location(latitude=20.0, altitude=90.0), customerType=1, gender='Female', email='julia.taylor@example.com', 2700, true, [Dish(id=12, name=Moussaka, desc="Baked eggplant with minced meat and béchamel sauce", price=65.0, restaurantId=32), Dish(id=54, name=Beef Empanadas, desc="Pastry filled with seasoned beef", price=35.0, restaurantId=32), Dish(id=33, name=Samosas, desc="Fried or baked pastry with a savory filling", price=25.0, restaurantId=32)], null, null

This output should be written to appropriate txt files, ready for further analysis or review.

Entity	Element	Type	Description
Class: DispatchSystem	dispatchSystem	DispatchSystem	The singleton you need to use.
	currentTimestamp	Long	The current time stamp, set to 3600s.
	availableDishes	List<Dish>	The list stores the dishes parsed from file.
	availableOrders	List<Order>	The list stores the orders parsed from file.
	dispatchedOrders	List<Order>	The list of orders dispatched.
	DispatchSystem()	Method	The contructor Task 1.
	getDishById(Long)	Method	Get dish by id for Task 4.
	checkDishesInRestaurant(Restaurant, Long[])	Method	Check if dishes in a restaurant, for Task 4.
	parseAccounts(String)	Method	Parse accounts from file, for Task 2.
	parseDishes(String)	Method	Parse dishes from file, for Task 3.
	parseOrders(String)	Method	Parse orders from file, for Task 4.
	getAvailablePendingOrders()	Method	Get available pending orders, for Task 5.
	getRankedPendingOrders(List<Order>)	Method	Get ranked pending orders, for Task 6.
	getAvailableRiders()	Method	Get available riders, for Task 7.
	matchTheBestTask(Order, List<Rider>)	Method	Match the best rider for a order, for Task 8.
	dispatchFirstRound()	Method	Dispatch orders to riders, for Task 9.
	writeOrders(String, List<Order>)	Method	Dump orders to file, for Task 9 and Task 10.
	getTimeoutDispatchedOrders()	Method	Get timeout dispatched orders, for Task 10.
Abstract Class: Account	id	Long	The ID of the account.
	accountType	String	Account type.
	name	String	Account name.
	contactNumber	String	Contact number.
	location	Location	Account location.
	accountManager	AccountManager	Static account manager.
	register()	Method	The abstract method for Task 2.
Class: Customer	customerType	Integer	Customer type.
	gender	String	Gender.
	email	String	Email.
	register()	Method	The abstract method for Task 2.
	getCustomerById(Long)	Method	Get account from manager.
Class: Restaurant	district	String	District.
	street	String	Street.
	dishes	List<Dish>	List of dishes.
	register()	Method	The abstract method for Task 2.
	addDish(Dish)	Method	Add dish to list.
	getRestaurantById(Long)	Method	Get account from manager.

Table 1: Required entities in this assignment.

Entity	Element	Type	Description
Class: Rider	gender	String	Rider gender.
	status	Integer	Rider status.
	userRating	Double	Rider's rating.
	monthTaskCount	Integer	Rider's month task count.
	register()	Method	The abstract method for Task 2.
	getRiderById(Long)	Method	Get account from manager.
Class: Location	latitude	Double	Location latitude.
	altitude	Double	Location altitude.
	distanceTo(Location)	Method	Get the distance between two locations.
Class: Dish	id	Long	Dish id.
	name	String	Dish name.
	desc	String	Dish description.
	price	BigDecimal	Dish price.
	restaurantId	Long	The restaurant the dish belongs to.
Class: Order	id	Long	Order id.
	status	Integer	Order status, see `Constants.java` in the skeleton.
	restaurant	Restaurant	Restaurant of the order.
	customer	Customer	Customer of the order.
	createTime	Long	The create time of the order.
	isPayed	Boolean	If the order is payed.
	orderedDishes	List<Dish>	Dishes ordered.
	rider	Rider	The rider of the order.
	estimatedTime	Double	The time elapsed in the delivery process.
	calculateEstimatedTime	Method	For Task 9
Class: Task	order	Order	The order associated with the task.
	rider	Rider	The rider associated with the task.

Table 2: Required entities in this assignment.

Entity	**Element**	**Type**	**Description**
Interface: PendingOrderRank	compare(Order, Order)	Method	Compare function inherited from Comparator<Order>.
Interface: TaskRank	compare(Task, Task)	Method	Compare function inherited from Comparator<Task>.
Class: (implements *TaskRank*) RiderMonthTaskCountRank	compare(Task, Task)	Method	Compare two tasks by rider's month task count.
Class: (implements *TaskRank*) RiderRatingRank	compare(Task, Task)	Method	Compare two tasks by rider's rating.
Class: (implements *TaskRank*) RiderToRestaurantRank	compare(Task, Task)	Method	Compare two tasks by distance.
Class: (impl. *PendingOrderRank*) CustomerPriorityRank	compare(Order, Order)	Method	Compare two orders by customer type.
Class: (impl. *PendingOrderRank*) OrderCreateTimeRank	compare(Order, Order)	Method	Compare two orders by create time.
Class: (impl. *PendingOrderRank*) RestaurantToCustomerDistanceRank	compare(Order, Order)	Method	Compare two orders by distance.

Table 3: Required entities in this assignment.